

LABORATORY ACTIVITY

AI-Assisted Framework-Based REST API Development

Backend Application Development for Information Technology

System Case

Student Information Management REST API

Laboratory Focus	Requirement
Development Mode	Backend only - no graphical frontend is required
Framework	Student choice using any appropriate backend framework
Primary Output	Working, secured, documented REST API
Database	Relational database with migrations/schema and seed data
Development Approach	AI-assisted software development with human review and accountability

The framework is a tool. The assessment focuses on API design, backend architecture, database quality, security, testing, documentation, and the student's ability to explain and modify the solution.

1. Laboratory Description

This laboratory activity requires students to design, develop, test, secure, and document a complete backend application using a RESTful API architecture. Students may choose the backend programming language, framework, ORM, database, and supporting tools that they consider appropriate for the problem, subject to the technical requirements stated in this activity.

The application will implement the backend of a Student Information Management System. It will expose structured HTTP endpoints for authentication, student records, academic programs, courses, academic terms, course offerings, enrollments, grades, and academic-record retrieval. A graphical frontend is intentionally excluded so that the laboratory concentrates on server-side engineering.

Artificial intelligence tools are encouraged as part of the development workflow. Students may use AI for planning, scaffolding, code generation, debugging, test generation, documentation, refactoring, and code review. However, every AI-generated artifact must be reviewed, tested, and understood before it is accepted into the project.

Expected Result

A reproducible REST API project that can be installed on another machine, connected to a database, tested through an API client, and explained during technical demonstration.

2. Learning Outcomes

Upon completion of the laboratory activity, students should be able to:

- Explain RESTful API principles and the relationship between HTTP methods, resources, status codes, and representations.
- Set up a backend project using a selected framework and relational database.
- Design normalized database tables, relationships, constraints, migrations, and seed data.
- Implement authentication and role-based authorization for protected resources.
- Develop CRUD endpoints and domain-specific endpoints using framework conventions.
- Validate incoming requests and return consistent success and error responses.
- Implement search, filtering, sorting, and pagination for collection endpoints.
- Apply backend security practices, including password hashing, secret management, and server-side authorization.
- Create and execute API tests using automated tests and an API client.
- Produce clear API and project documentation using OpenAPI/Swagger or an equivalent method.
- Use AI tools responsibly as development assistants while maintaining technical ownership of the submitted work.

3. AI-Assisted Development Policy

AI Use is Encouraged

Students are encouraged to use modern AI-assisted development tools. The goal is to practice contemporary software engineering in which AI accelerates work while the developer remains responsible for architecture, correctness, security, testing, and explanation.

3.1 Permitted Uses of AI

- Requirements analysis and decomposition

- Framework setup and configuration guidance
- Database and ERD design review
- Route, controller, service, repository, and model scaffolding
- Validation rules and error-handling patterns
- Debugging and root-cause analysis
- Test-case and test-data generation
- OpenAPI/Swagger documentation assistance
- Security and code-review checklists
- Refactoring and performance suggestions
- README and technical documentation drafting

3.2 AI Accountability

The use of AI does not transfer responsibility for the final code. Students must verify all generated outputs and must be able to explain, debug, and modify the implementation during the laboratory demonstration.

- Do not submit code that you cannot explain.
- Do not accept AI-generated database or security code without reviewing it.
- Do not paste real passwords, private tokens, or confidential credentials into AI tools.
- Run tests after meaningful AI-generated changes.
- Prefer small, reviewable prompts and commits instead of asking an AI tool to generate the entire application at once.
- Record major AI contributions in the README or development notes when required by the instructor.

3.3 Recommended AI-Assisted Workflow

Step	Developer Action
1. Understand	Read the requirement and identify the business rule before prompting.
2. Plan	Define the resource, data model, endpoint, validation, security, and tests.
3. Prompt	Ask AI for a specific, bounded task using project context.
4. Generate	Allow AI to propose code, tests, documentation, or fixes.
5. Review	Read the generated output and compare it with framework conventions.
6. Test	Run automated tests and API-client requests.
7. Improve	Refactor, correct, secure, and optimize the implementation.
8. Document	Update README, API docs, and development notes.

4. Framework and Technology Freedom

Students may select any suitable backend framework. The selected framework does not determine the grade; the quality of the implementation does.

Language	Examples of Appropriate Frameworks
PHP	Laravel, Symfony
JavaScript / TypeScript	Express.js, Fastify, NestJS
Python	Django REST Framework, FastAPI, Flask
Java	Spring Boot
C#	ASP.NET Core Web API
Go	Fiber, Gin, Echo
Ruby	Ruby on Rails
Kotlin	Ktor, Spring Boot
Other	Any appropriate framework approved by the instructor

4.1 Database Choices

A relational database is required. Appropriate choices include PostgreSQL, MySQL, MariaDB, SQL Server, or SQLite for local development. Database access should use the framework's ORM/query layer or another safe parameterized method.

5. System Scenario

Develop a Student Information Management REST API that can be consumed by a future web, mobile, or desktop client. The API must centralize student and academic information and enforce access according to user roles.

5.1 Required User Roles

Role	Minimum Responsibility
Administrator	Full system administration and access to all managed resources.
Registrar / Staff	Manage students, academic records, programs, courses, and enrollments as permitted.
Instructor	View assigned course offerings and manage grades for authorized enrollments.
Student	View only the student's own profile, enrollments, and grades.

6. Required Domain Modules

Module	Minimum Scope
Authentication	Login, logout/token invalidation where applicable, current authenticated user.
Users and Roles	User accounts, roles, status, authorization rules.
Students	Student profile, student number, program, year level, status, contact data.
Programs	Academic program code, name, description, status.
Courses	Course code, title, units, description, status.
Academic Terms	Academic year, semester/term, dates, status.
Course Offerings	Course, term, instructor, section, schedule, capacity, status.
Enrollments	Student registration in a course offering with status.
Grades	Grades linked to enrollment with remarks/status.
Academic Record	Aggregated student record grouped by academic term.

7. Minimum Data Model

The following fields are minimum recommendations. Students may refine the schema where justified, but relationships, keys, and constraints must remain correct.

Entity	Minimum Fields
users	id, name, email, password_hash, role/role_id, status, created_at, updated_at
students	id, student_number, first_name, middle_name, last_name, suffix, birth_date, email, contact_number, address, program_id, year_level, status, timestamps
programs	id, code, name, description, status, timestamps
courses	id, course_code, course_title, description, units, status, timestamps
academic_terms	id, academic_year, semester/term, start_date, end_date, status, timestamps
course_offerings	id, course_id, academic_term_id, instructor_id, section, schedule, room, capacity, status, timestamps
enrollments	id, student_id, course_offering_id, enrollment_date, status, timestamps
grades	id, enrollment_id, midterm_grade, final_grade/final_rating, remarks, timestamps

7.1 Required Relationships

- One program has many students.

- One course may have many course offerings.
- One academic term may contain many course offerings.
- One instructor may handle many course offerings.
- One student may have many enrollments.
- One course offering may have many enrollments.
- An enrollment may have one grade record or an equivalent normalized grade structure.

7.2 Data Integrity Requirements

- `student_number` must be unique.
- `course_code` must be unique.
- Foreign keys must protect valid relationships.
- Duplicate enrollment in the same course offering must be prevented.
- Appropriate indexes should support frequently searched columns.
- Schema changes should use migrations or an equivalent reproducible mechanism.

8. REST API Design Standards

Use API versioning and resource-oriented URLs. A typical base path is `/api/v1`. Resource URLs should use nouns, while the HTTP method expresses the operation.

Operation	Recommended Pattern
List students	GET <code>/api/v1/students</code>
Create student	POST <code>/api/v1/students</code>
Retrieve student	GET <code>/api/v1/students/{id}</code>
Replace/update student	PUT <code>/api/v1/students/{id}</code>
Partial update	PATCH <code>/api/v1/students/{id}</code>
Delete/deactivate	DELETE <code>/api/v1/students/{id}</code>

Avoid RPC-style resource names

Prefer GET `/students` and POST `/students`. Avoid endpoints such as `/getStudents`, `/createStudent`, or `/deleteStudent` unless the operation is truly an action that cannot be expressed as a resource.

8.1 Required Endpoint Groups

Authentication

- POST `/api/v1/auth/login`
- POST `/api/v1/auth/logout` or equivalent token invalidation
- GET `/api/v1/auth/me`

Students

- GET `/api/v1/students`
- POST `/api/v1/students`
- GET `/api/v1/students/{id}`
- PUT or PATCH `/api/v1/students/{id}`
- DELETE `/api/v1/students/{id}`

Programs

- GET/POST `/api/v1/programs`
- GET/PUT/PATCH/DELETE `/api/v1/programs/{id}`

Courses

- GET/POST `/api/v1/courses`
- GET/PUT/PATCH/DELETE `/api/v1/courses/{id}`

Academic Terms

- GET/POST `/api/v1/academic-terms`
- GET/PUT/PATCH/DELETE `/api/v1/academic-terms/{id}`

Course Offerings

- GET/POST `/api/v1/course-offerings`
- GET/PUT/PATCH/DELETE `/api/v1/course-offerings/{id}`

Enrollments

- GET/POST /api/v1/enrollments
- GET/PATCH/DELETE /api/v1/enrollments/{id}
- GET /api/v1/students/{id}/enrollments
- GET /api/v1/course-offerings/{id}/students

Grades

- GET/POST /api/v1/grades
- GET/PUT/PATCH /api/v1/grades/{id}
- GET /api/v1/students/{id}/grades

Academic Record

- GET /api/v1/students/{id}/academic-record

8.2 HTTP Status Codes

Code	Expected Use
200 OK	Successful retrieval or update with response content.
201 Created	A resource was successfully created.
204 No Content	Successful operation without response body when appropriate.
400 Bad Request	Malformed or invalid request outside normal validation semantics.
401 Unauthorized	Authentication is missing or invalid.
403 Forbidden	Authenticated user lacks required permission.
404 Not Found	Requested resource does not exist.
409 Conflict	Duplicate/conflicting state where appropriate.
422 Unprocessable Content	Request validation failed.
500 Internal Server Error	Unexpected server failure; details must not leak sensitive internals.

9. Response and Error Consistency

Establish a consistent JSON response format. Framework conventions may be used, but the API must be predictable and documented.

Example Success Response

```
{
  "success": true,
  "message": "Student retrieved successfully.",
  "data": { "id": 1, "student_number": "2026-00001" }
}
```

Example Validation Error

```
{
  "success": false,
```

```
"message": "Validation failed.",
"errors": { "email": ["A valid email address is required."] }
}
```

10. Validation Requirements

- student_number: required and unique.
- first_name and last_name: required.
- email: valid email format when supplied.
- program_id: must reference an existing program.
- course_id, academic_term_id, instructor_id, student_id, and course_offering_id: must reference valid records.
- year level and status values: restricted to documented valid values.
- grade values: must follow the grading range/rules defined by the implementation.

Validation must occur on the server. Invalid input must not be persisted simply because a client sends it.

11. Search, Filtering, Sorting, and Pagination

Feature	Example
Search	GET /api/v1/students?search=dela
Filter	GET /api/v1/students? program_id=1&year_level=3&status=ACTIVE
Sort	GET /api/v1/students?sort=last_name or another documented convention
Pagination	GET /api/v1/students?page=2&per_page=20

Collection endpoints must not return unbounded datasets. Pagination metadata should include enough information for a client to navigate results, such as current page, page size, total records, and last page or equivalent cursor information.

12. Authentication and Authorization

The API must implement secure authentication and server-side authorization. Authentication answers who the user is; authorization determines what that user is allowed to do.

12.1 Authentication

- Passwords must be hashed using an appropriate framework-supported password-hashing mechanism.
- Protected routes must require a valid authenticated session or token.
- The API must provide a current-user endpoint or equivalent mechanism.
- Logout or token invalidation should be implemented where supported by the authentication strategy.

12.2 Authorization

Role	Minimum Access Rule
Administrator	May manage all required resources.
Registrar / Staff	May manage student and academic records according to configured permissions.
Instructor	May view assigned offerings and modify grades only for authorized enrollments.
Student	May view only the student's own profile, enrollment, and grade information.

Object-Level Authorization

A student must not be able to retrieve another student's confidential academic record merely by changing an ID in the URL. Authorization must be enforced on the server for the actual requested resource.

13. Backend Security Requirements

- Use environment variables or a secure configuration mechanism for secrets.
- Do not commit the real .env file, database password, private keys, or tokens to a public repository.
- Use parameterized queries, ORM/query-builder safeguards, and proper validation to prevent SQL injection.
- Do not expose raw database exceptions, stack traces, or sensitive implementation details in normal production responses.
- Do not return password hashes, authentication secrets, or unnecessary sensitive fields in API responses.
- Apply authorization to every protected action, not only to menu or client-side behavior.
- Configure CORS deliberately when browser clients are expected; do not use unrestricted production settings without justification.
- Log useful operational errors without logging passwords or private tokens.

14. Environment and Reproducibility

The project must be reproducible on another machine. Include a safe .env.example or equivalent configuration template, migration/schema files, seeders/fixtures, dependency manifests, and installation instructions.

Example Configuration Variables

```
APP_ENV=development
APP_PORT=8000
```

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=student_api
DB_USER=app_user
DB_PASSWORD=...
AUTH_SECRET=...
```

15. Required Test Data

Entity	Minimum Demonstration Records
Users	5
Programs	3
Students	100
Courses	20
Academic Terms	2
Course Offerings	20
Enrollments	200
Grades	100

Use seeders, factories, fixtures, scripts, or equivalent tools. Manually inserting every demonstration record is discouraged.

16. Testing Requirements

Testing must include successful and unsuccessful requests. Automated tests are strongly encouraged and should cover the most important security and business flows.

Area	Minimum Test Cases
Authentication	Valid login; invalid password; missing/invalid authentication.
Students	Create; retrieve; update; duplicate student number; invalid email; not found.
Authorization	Permitted administrator action; forbidden student/instructor actions; object-level access control.
Enrollment	Valid enrollment; invalid references; duplicate enrollment prevention.
Grades	Valid grade; invalid enrollment; unauthorized grade modification.
Collections	Search; filtering; sorting; pagination behavior.

16.1 Recommended Automated Test Tools

Use the test framework appropriate to the chosen technology, such as Pest/PHPUnit, Vitest/Jest, Pytest, JUnit, xUnit, or Go testing. At minimum, automated tests should exercise authentication, student creation, validation, authorization, enrollment, and grade submission.

17. API Documentation

Complete API documentation is mandatory. Swagger UI/OpenAPI is preferred, but equivalent documentation may be accepted when it clearly describes the contract.

- Endpoint path and HTTP method
- Description and purpose
- Authentication requirement
- Path/query parameters
- Request body schema and example
- Successful response schema and example
- Validation/error response examples
- Relevant HTTP status codes

A documented route such as `/api/docs` is recommended where the framework supports it.

18. API Client Collection

Prepare a Postman, Insomnia, Bruno, or equivalent API collection that demonstrates the major endpoints and includes both successful and failed requests. Organize requests by resource group: Authentication, Students, Programs, Courses, Academic Terms, Course Offerings, Enrollments, Grades, and Academic Records.

19. Laboratory Development Phases

Phase	Required Work	Checkpoint
1. Project Setup	Select framework; initialize repository; configure environment and database.	API server starts and database connection succeeds.
2. Database Design	Create ERD, schema/migrations, constraints, relationships, and seeders.	Database can be recreated from project files.
3. Authentication	Implement login, password hashing, protected routes, current user.	Protected endpoint rejects unauthenticated request.
4. Core Resources	Implement Programs, Students, Courses, Academic Terms.	CRUD requests work with validation.
5. Academic Transactions	Implement Course Offerings, Enrollments, Grades, Academic Record.	Relationships and domain operations work.
6. Advanced API Features	Implement search, filtering, sorting, pagination, consistent errors.	Collection endpoints meet requirements.
7. Authorization	Apply permissions and object-level access rules.	Unauthorized roles receive 403 or equivalent.
8. Documentation	Complete README, OpenAPI/Swagger, ERD, API collection.	Another developer can understand/setup the API.
9. Testing	Run positive, negative, validation, security, and automated tests.	Critical tests pass.
10. Final Demonstration	Deploy/start locally and demonstrate API end-to-end.	All mandatory acceptance cases are shown.

20. Required Final Outputs

1. Complete backend source code.
2. Database migration/schema files.
3. Seeders, fixtures, factories, or data-generation scripts.
4. Safe .env.example or equivalent configuration template.
5. Entity Relationship Diagram (ERD).
6. OpenAPI/Swagger or equivalent API documentation.
7. Postman/Insomnia/Bruno or equivalent API collection.
8. Automated test suite and/or test evidence.
9. README.md with installation, configuration, migration, seeding, running, authentication, and testing instructions.
10. Git repository with meaningful development history.
11. Short technical documentation describing architecture, major design decisions, security approach, and AI-assisted development workflow.

21. README Minimum Content

- Project title and description
- Selected technology stack
- Prerequisites
- Installation instructions
- Environment configuration

- Database setup
- Migration/schema instructions
- Seeder/fixture instructions
- How to start the API
- Authentication instructions
- API documentation location
- How to run tests
- Development test accounts if used, without exposing real credentials
- Summary of how AI tools were used and how generated outputs were verified

22. Mandatory Acceptance Demonstration

The final demonstration must show the following actions using Swagger, Postman, Insomnia, Bruno, REST Client, curl, or another API testing tool:

#	Demonstration
1	Start the REST API and connect to the database.
2	Authenticate successfully.
3	Show a protected endpoint rejecting an unauthenticated request.
4	Create a program.
5	Create a valid student.
6	Show validation rejecting an invalid or duplicate student.
7	Retrieve and update a student.
8	Search and filter student records.
9	Show pagination and sorting.
10	Create a course and academic term.
11	Create a course offering.
12	Enroll a student.
13	Prevent or handle duplicate enrollment correctly.
14	Encode or update an authorized grade.
15	Retrieve a student academic record.
16	Show a forbidden request for an unauthorized role.
17	Show a 404/not-found case.
18	Display API documentation.
19	Run or show automated test results.
20	Explain one AI-assisted code contribution and demonstrate understanding of it.

23. Minimum Acceptance Criteria

- API server runs without critical errors.
- Database is integrated and reproducible from project files.
- Authentication and protected routes work.
- Role-based and object-level authorization are enforced.
- Required CRUD and academic transaction endpoints work.
- Validation prevents invalid data from being persisted.
- Search, filtering, sorting, and pagination work.
- Correct HTTP status codes are used.
- Errors are returned consistently without leaking sensitive details.
- API documentation and API-client collection are complete.
- At least the required critical automated tests or equivalent testing evidence are present.
- The student can explain and modify the implementation, including AI-generated portions.

24. Grading Rubric

Criterion	Points
REST API design and correct HTTP usage	15
Database design, relationships, migrations, and data integrity	15
Core CRUD and academic transaction functionality	15
Authentication and authorization	15
Validation and error handling	10
Search, filtering, sorting, and pagination	10
API documentation and reproducibility	5
Testing and reliability	5
Code organization and maintainability	5
Technical demonstration, AI accountability, and understanding	5
TOTAL	100

24.1 Rubric Interpretation

REST API Design - 15

Resource-oriented endpoints, correct methods, versioning, status codes, response consistency, and documented contracts.

Database Design - 15

Normalized structure, appropriate relationships, keys, constraints, migrations/schema, seed data, and integrity.

Core Functionality - 15

Required resources and academic transactions function correctly without hardcoded/static data.

Authentication and Authorization - 15

Secure authentication, password hashing, protected routes, role permissions, and object-level authorization.

Validation and Error Handling - 10

Robust server-side validation, duplicate/conflict handling, not-found behavior, and consistent errors.

Advanced Collection Features - 10

Search, filtering, sorting, pagination, and efficient collection endpoints.

Documentation - 5

README, setup, API documentation, ERD, and API client collection are complete and usable.

Testing - 5

Positive, negative, validation, authorization, and critical automated test evidence.

Code Quality - 5

Clear organization, naming, framework conventions, separation of concerns, and maintainability.

Demonstration and AI Accountability - 5

Student can explain architecture, code, database, security, testing, and AI-assisted contributions; can perform a small modification or debugging task when asked.

25. Critical Deficiencies

Significant Deductions

A submission may receive major deductions if it has no working REST API, no real database integration, no authentication, plaintext passwords, missing server-side authorization, static/hardcoded API data, no validation, no API demonstration, or if the student cannot explain major source code and AI-generated portions.

26. Optional Enhancements

- Refresh-token strategy or secure session rotation
- Email verification and password reset
- Rate limiting
- Soft delete and recovery
- Audit trail/activity logs
- Dockerized development environment
- CI/CD pipeline
- Redis caching
- CSV import/export
- PDF academic record generation
- Health/readiness endpoints
- Role and permission administration
- Integration tests and performance tests

Bonus features do not replace incomplete core requirements.

27. Final Development Principle

AI Accelerates Development; the Developer Owns the Result

Use AI to reduce repetitive work, explore alternatives, debug faster, and improve documentation. Do not use AI as

a substitute for understanding. Every endpoint, schema decision, security rule, and test result remains the responsibility of the developer.

28. Final Submission Checklist

- Backend source code runs successfully.
- No frontend is required for grading.
- Database schema/migrations and seed data are included.
- Authentication and authorization are functional.
- All required resources and academic transaction endpoints are implemented.
- Search, filtering, sorting, and pagination work.
- Validation and error handling are consistent.
- Secrets are excluded from the repository.
- README and API documentation are complete.
- API client collection is included.
- Tests have been executed and evidence is available.
- AI-assisted contributions have been reviewed and can be explained.

END OF LABORATORY ACTIVITY